

Força bruta, busca exaustiva e backtrack recursivo

9.1 Força bruta

Quando projetamos um algoritmo que **decorre de forma simples e direta da definição** do problema, dizermos que estamos utilizando força bruta. Chamamos desta forma pois **não aplicamos esforço mental para projetar um algoritmo eficiente**, deixando o computador com o esforço de resolver o problema de forma menos eficiente. **Geralmente os problemas podem ser resolvidos de forma mais eficiente** que o algoritmo obtido por força bruta.

Por exemplo, para **calcular** b^n , onde b é não nulo e n é inteiro não negativo, a forma “direta” é multiplicar n cópias de b , ou seja

$$b^n = \underbrace{b \cdot b \cdot \dots \cdot b}_n$$

Se considerarmos a multiplicação com tempo constante, este algoritmo será $O(n)$. Usando o fato de que $b^n = b^{\lfloor n/2 \rfloor} \cdot b^{\lfloor n/2 \rfloor}$ se n é par, e $b^n = b^{\lfloor n/2 \rfloor} \cdot b^{\lfloor n/2 \rfloor} \cdot b$ se n é ímpar, podemos projetar um algoritmo recursivo mais eficiente com tempo $T(n) = T(n/2) + \Theta(1) \in \Theta(\log n)$.

Outro exemplo é o **problema de ordenação**, cuja estratégia direta (bolha ou ordenação por seleção, por exemplo) produz algoritmos $\Theta(n^2)$, mas existem algoritmos mais sofisticados com tempo $\Theta(n \log n)$.

Apesar de raramente produzir um algoritmo eficiente, o projeto por força bruta é importante, pois

- É mais **provável que o problema admita solução por força bruta** que o contrário. Isto não é verdade para as outras técnicas de projeto.
- Para **alguns problemas** importantes produz algoritmos com **eficiência satisfatória**, como no problema de ordenação.
- O **esforço adicional** para produzir algoritmo mais eficiente **pode ser desnecessário** quando o algoritmo for usado poucas vezes e com tamanhos de entrada que produzem tempo de execução aceitável.
- Pode servir como **ponto de partida** para soluções mais sofisticadas.
- Pode **ajudar a testar** outros algoritmos, pois conseguimos executar e produzir soluções corretas para instâncias pequenas.

9.2 Busca exaustiva

Muitos problemas importantes consistem em encontrar um elementos com determinada propriedade em um **domínio que cresce exponencialmente com o tamanho da entrada**. Geralmente nestes problemas os elementos são objetos combinatórios, como **permutações, combinações e subconjuntos**. Muitos destes problemas são de **otimização**, ou seja, desejamos encontrar um elemento que maximiza ou minimiza um determinado critério.

Uma **busca exaustiva** é simplesmente uma abordagem **força bruta para um problema combinatório**. Consiste em gerar todos os objetos do domínio (que satisfazem todas as restrições) em busca de um elemento que satisfaz a propriedade desejada (como por exemplo maximizar/minimizar uma função objetivo). Como o domínio de objetos combinatórios cresce exponencialmente com o tamanho da entrada, esta abordagem **só funciona para entradas pequenas**.

Por exemplo, considere o problema de encontrar um **caminho de comprimento mínimo entre dois nós s e t** em um grafo direcionado com pesos não negativos nas arestas. Vimos que este problema pode ser resolvido através do algoritmo de **Dijkstra**. É um **problema combinatório**, pois um caminho é um subconjunto das arestas. É também um **problema de otimização**, pois desejamos encontrar um caminho cuja soma dos pesos das arestas seja mínimo. Para resolver por **busca exaustiva**, geramos todo subconjunto A de arestas, e testamos se A produz um caminho de s até t . Se A produz um caminho de s e t , e possui o menor custo dentre todos os caminhos de s até t avaliados até então, guardamos o conjunto A . Ao final da exploração retornamos a melhor solução encontrada.

Para resolver um problema por busca exaustiva precisamos saber como gerar todas as permutações/combinações/subconjuntos, como veremos a seguir.

9.3 Backtrack recursivo

Um algoritmo de **backtrack recursivo** avaliar recursivamente todas as possíveis soluções para o problema. Portanto, se o problema é de otimização, o backtrack recursivo realiza uma busca exaustiva.

9.3.1 Ex.: geração de permutações

Algoritmo: **permutações**(S , prefixo = [])

Entrada: S : conjunto contendo os elementos que devem ser permutados. prefixo: lista com os elementos que devem ser impressos antes de cada permutação de S .

Saída : Para cada permutação p de S , imprime prefixo seguido da permutação p .

```
1 if len(S) = 0 then
2     // Caso base: sem elemento para permutar, basta imprimir o prefixo.
3     print(prefixo)
4 else
5     foreach  $e$  in  $S$  do
6         // Remove  $e$  de  $S$  e coloca no final de prefixo.
7          $S.remove(e)$ 
8         prefixo.append( $e$ )
9         permutações( $S$ , prefixo)
10        // Recupera estado original de  $S$  e prefixo.
11         $S.add(e)$ 
12        prefixo.pop()
```

Exemplo de execução:

```
> permutacoes({1, 2, 3})
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]
```

Complexidade

Como o algoritmo imprime $n!$ permutações, temos que a complexidade é $\Omega(n!)$, e portanto tem crescimento maior que exponencial.

9.3.2 Ex.: geração de subconjuntos

Algoritmo: <code>subconjuntos</code> (<i>S</i> , prefixo = [])	
Entrada: <i>S</i> : lista contendo os elementos do conjunto de onde devemos extrair os subconjuntos. <code>prefixo</code> : lista com os elementos que devem ser impressos antes de cada subconjunto de <i>S</i> .	
Saída : Para cada subconjunto <i>A</i> de <i>S</i> , imprime <code>prefixo</code> e em seguida os elementos de <i>A</i> .	
1	if len(<i>S</i>) = 0 then
2	// Caso base: conjunto <i>S</i> sem elementos, basta imprimir o prefixo.
3	print(prefixo)
4	else
5	// Remove um elemento <i>e</i> qualquer de <i>S</i> .
6	<i>e</i> = <i>S</i> .pop()
7	// Gera todos os subconjuntos que NÃO POSSUEM o elemento <i>e</i> .
8	<code>subconjuntos</code> (<i>S</i> , prefixo)
9	// Gera todos os subconjuntos que POSSUEM o elemento <i>e</i> .
10	// (basta acrescentar <i>e</i> ao prefixo)
11	prefixo.append(<i>e</i>)
12	<code>subconjuntos</code> (<i>S</i> , prefixo)
13	// Recupera estado original de <i>S</i> e <code>prefixo</code> .
14	<i>S</i> .append(<i>e</i>)
15	prefixo.pop()

Exemplo de execução:

```
> subconjuntos([1, 2, 3])
[]
[1]
[2]
[2, 1]
[3]
[3, 1]
[3, 2]
[3, 2, 1]
```

Complexidade

Seja *n* o número de elementos em *S*. Então,

$$T(n) = 2T(n - 1) + \Theta(1) \in \Theta(2^n).$$

9.3.3 Ex.: problema da mochila binário

Algoritmo: <code>mochila</code> (<i>itens</i> , capacidade)	
Entrada: <i>itens</i> : lista de tuplas (id, peso, valor), contendo o id, peso e valor de cada item. capacidade: máxima soma dos pesos dos itens selecionados.	
Saída : Retorna tupla (valor, ids) com o valor ótimo e um conjunto de itens que maximiza o valor total sem violar a capacidade.	
// Caso base: se não tem itens, solução é o conj. vazio com valor zero.	
1	if len(<i>itens</i>) = 0 then
2	return (0, set())
// Remove um item qualquer da lista de itens (item <code>id_item</code>).	
3	(<code>id_item</code> , <code>peso_item</code> , <code>valor_item</code>) = <i>itens</i> .pop()
// Resolve recursivamente SEM COLOCAR <code>id_item</code> na solução.	
4	(<code>valor_sol1</code> , <code>ids_sol1</code>) = <code>mochila</code> (<i>itens</i> , capacidade)
// Se <code>id_item</code> cabe na mochila, resolve COLOCANDO ele na solução.	
5	<code>valor_sol2</code> = $-\infty$
6	if <code>peso_item</code> ≤ capacidade then
7	(<code>valor_sol2</code> , <code>ids_sol2</code>) = <code>mochila</code> (<i>itens</i> , capacidade - <code>peso_item</code>)
// Recupera estado original da lista de itens.	
8	<i>itens</i> .append((<code>id_item</code> , <code>peso_item</code> , <code>valor_item</code>))
// Retorna a solução com maior valor, considerando o valor do item.	
9	if <code>valor_sol1</code> ≥ <code>valor_sol2</code> + <code>valor_item</code> then
10	return (<code>valor_sol1</code> , <code>ids_sol1</code>)
11	else
12	return (<code>valor_sol2</code> + <code>valor_item</code> , <code>ids_sol2</code> { <code>id_item</code> })

Exemplo de execução:

```
> itens = [(1,6,30), (2,3,14), (3,4,16), (4,2,9)]
> print(mochila(itens, 10))
(46, {1, 3})
```

Complexidade

Seja *n* o número de itens. Então,

$$T(n) = 2T(n - 1) + \Theta(1) \in \Theta(2^n).$$

9.4 Próxima permutação/subconjunto em ordem lexicográfica (opcional)

Algoritmo: próxima_r_combinação($n, a[1..r]$)
Entrada: Lista $a[1..r]$ contendo uma r-combinação de $\{1, 2, \dots, n\}$ em ordem crescente ($a[1] < a[2] < \dots < a[r]$). A r-combinação a não é a última ($u = [n - r + 1, \dots, n]$). Saída : Modificações na lista $a[1..r]$ de modo a torná-la a próxima na ordem lexicográfica, e com elementos em ordem crescente. <pre> // Passo 1: localiza maior i tal que a[i] difere de u[i] 1 i ← r 2 while a[i] = n - r + i do 3 i ← i - 1 // Passo 2: incrementa a[i] 4 a[i] ← a[i] + 1 // Passo 3: coloca os sucessores de a[i] nas posições maiores que i 5 for j de i + 1 até r do 6 a[j] ← a[j] + j - i </pre>

9.4.4 Exemplo: problema do caixeiro viajante

No problema do caixeiro viajantes temos um grafo não direcionado com pesos não negativos $d[u][v]$ em cada arestas (u, v) , e desejamos **determinar o percurso com menor peso total que passa por todos os vértices e retorna para o vértice inicial**. Por exemplo, suponha que cada vértice representa uma cidade, e o peso de cada aresta (u, v) representa a duração da viagem entre as cidades u e v . Neste caso a solução será um trajeto que passa por todas as cidades gastando o menor tempo total.

Indexando os vértices de 1 até n , **cada percurso é uma permutação** $a[1..n]$ de $\{1, 2, \dots, n\}$. O **custo da permutação** $a[1..n]$ vale

$$\text{custo}(a) = d[a[n]][a[1]] + \sum_{j=2}^n d[a[j-1]][a[j]].$$

Para resolver por **busca exaustiva** basta **gerar todas as permutações** das n cidades. Para **reduzir as simetrias** (soluções idênticas), podemos fixar que a primeira cidade sempre vale 1, ou seja, $a[1] = 1$. Neste caso basta iniciar na permutação $[1, 2, \dots, n]$ e explorar as próximas $(n - 1)!$ permutações na ordem lexicográfica.

Algoritmo: caixeiro_viajante($d[1..n][1..n]$)
Entrada: Cada elemento $d[i][j]$ da matriz d fornece a distância da cidade i para a cidade j. As distâncias são reais não negativos. Saída : Retorna uma permutação das cidades de custo mínimo. <pre> 1 a[1..n] ← [1, 2, ..., n] // (1a permutação das n cidades) O(n) 2 menor_custo ← ∞ 3 for i de 1 até (n - 1)! do // O((n - 1)!) 4 // calcula o custo da permutação a 5 custo ← d[a[n]][a[1]] // O((n - 1)!) 6 for j de 2 até n do // O(n(n - 1)!) = O(n!) 7 custo ← custo + d[a[j-1]][a[j]] // O(n!) 8 // armazena a se tiver o menor custo 9 if custo < menor_custo then // O((n - 1)!) 10 menor_custo ← custo // O((n - 1)!) 11 melhor_solucao ← a // O((n - 1)!) 12 proxima_permutacao(a) // O(n(n - 1)!) = O(n!) 13 return melhor_solucao // O(1) </pre>

Observando as complexidades das linhas indicadas no pseudocódigo, concluímos que o algoritmo tem **complexidade** $O(n!)$.

Leitura complementar

Busca exaustiva: Seção 7.1 do Skiena.

Geração de permutações e combinações: Seção 5.6 do Rosen.